

Kiến trúc SOA & Microservices

Đặng Ngọc Hùng

Mục lục

7. Quản lý Dữ liệu — Database per Service & CQRS	4
7.1. Database-per-Service — Nguyên tắc Data Ownership	5
7.1.1. Vấn đề: shared database phá vỡ service independence	5
7.1.2. Database-per-service pattern	6
7.1.3. Industry Case Study: Uber — Domain-Oriented Microservice Architecture (DOMA)	8
7.1.4. CAP Theorem — Hiểu đúng giới hạn	8
7.1.5. Caching Strategies — Giảm load và latency	10
7.2. Chiến lược tách Database từ Monolith	11
7.2.1. Vấn đề: tách service dễ, tách data khó	11
7.2.2. Năm chiến lược tách database	11
7.2.3. Áp dụng cho KBM	12
7.3. Data Duplication vs Coupling — Khi nào chấp nhận duplicate?	12
7.3.1. Vấn đề: cần data từ service khác	12
7.3.2. Ba chiến lược truy cập data xuyên service	12
7.3.3. Khi nào chấp nhận duplication?	14
7.4. CQRS — Command Query Responsibility Segregation	15
7.4.1. Từ CQS đến CQRS	15
7.4.2. Vấn đề: cùng model cho read và write không đủ	16
7.4.3. CQRS pattern	16
7.4.4. Khi nào cần CQRS?	17
7.4.5. Ví dụ: Leaderboard trong Contest Mode - Tại sao CQRS bất đồng bộ (qua Kafka) không phù hợp cho Live Leaderboard?	17
7.5. Event Sourcing — Lưu Events thay vì State	19
7.5.1. Vấn đề: mất lịch sử khi chỉ lưu state	19
7.5.2. Event Sourcing pattern	19
7.5.3. Snapshot Pattern — Giải quyết Performance của Event Replay	21
7.5.4. Projection Rebuilds — Tạo lại Views từ Events	22
7.5.5. Event Schema Evolution — Versioning Events	24
7.5.6. Khi nào dùng Event Sourcing?	24
7.6. Case Study: KBM	25
7.6.1. Hiện trạng	25
7.6.2. Từ hiện trạng đến kiến trúc mục tiêu	26
7.6.3. Đề xuất migration path	26
7.7. Tổng kết	27
7.8. Đọc thêm	28
Tài liệu tham khảo	29

7.

CHƯƠNG 7.

Quản lý Dữ liệu — Database per Service & CQRS

“Duplication is far better than coupling. Each service should own its data and make independent decisions about what to store.”

— Sam Newman, *Monolith to Microservices* [4b]

MỤC TIÊU HỌC TẬP

- Hiểu nguyên tắc **database-per-service** và tại sao data ownership là nền tảng của microservices
- Nắm được các chiến lược tách database từ monolith — từ shared database đến database-per-service
- Phân biệt khi nào data duplication là chấp nhận được và khi nào nó là vấn đề
- Hiểu CQRS (Command Query Responsibility Segregation) và khi nào nó cần thiết
- Nắm tổng quan Event Sourcing — ưu/nhược điểm và khi nào nên áp dụng
- Phân tích bài toán cross-service queries trong KBM

i Bản đồ khái niệm: Ch.5 → Ch.6 → Ch.7

Ba chương cuối Phần II nối với nhau theo một mạch: **Ch.5** cho chúng ta cách các service nói chuyện bất đồng bộ qua message/event; **Ch.6** dùng chính các message đó để giữ tính nhất quán khi một nghiệp vụ trải trên nhiều service (Saga); và **Ch.7** (chương này) trả lời câu hỏi sâu hơn: nếu mỗi service *sở hữu database riêng*, thì lưu trữ, truy vấn và đồng bộ dữ liệu giữa chúng ra sao? Nói gọn: Ch.5 = “gửi tin”, Ch.6 = “phối hợp giao dịch”, Ch.7 = “quản lý dữ liệu của từng service”. Nếu khối lượng kiến thức quá lớn, độc giả chỉ cần ghi nhớ một nguyên lý cốt lõi cho chương này: **mỗi service quản lý dữ liệu độc lập, và chỉ chia sẻ qua API hoặc event — không bao giờ chung bảng.**

7.1. Database-per-Service — Nguyên tắc Data Ownership

Để đạt được tính tự chủ, mỗi dịch vụ cần quản lý một không gian lưu trữ độc lập, tương tự như việc các khoa trong trường đại học tự quản lý hồ sơ sinh viên của khoa đó.

7.1.1. Vấn đề: shared database phá vỡ service independence

Trong monolith, toàn bộ ứng dụng truy cập một database duy nhất — đơn giản, nhất quán, và transactions ACID hoạt động tự nhiên. Khi chuyển sang kiến trúc microservices, nhiều nhóm phát triển vẫn giữ nguyên cơ sở dữ liệu dùng chung vì sự tiện lợi — mọi service vẫn đọc/ghi cùng bảng. Thoạt nhìn, đây có vẻ là giải pháp tốt: không cần thay đổi data layer, không cần xử lý eventual consistency.

Tuy nhiên, bài toán data trong distributed systems bị ràng buộc bởi một giới hạn lý thuyết quan trọng — **CAP Theorem** (Consistency, Availability, Partition tolerance): trong hệ thống phân tán, khi network partition xảy ra, hệ thống chỉ có thể chọn *hoặc* consistency *hoặc* availability, không thể có cả hai [7, Ch.9]. Newman trong [4a, Ch.12] giải thích: đây là lý do microservices *thường* chấp nhận eventual consistency cho các bản sao dữ liệu liên service — nhưng mỗi operation vẫn phải tự chọn hành vi khi partition (chấp nhận trả lời dữ liệu có thể cũ, hay từ chối phục vụ để giữ nhất quán) — và tại sao thiết kế data management cần cân nhắc kỹ trade-off giữa consistency và availability cho *từng use case cụ thể*.

Shared database **phá vỡ nguyên lý cốt lõi nhất** của microservices: independent deployability. Newman trong [4b, Ch.4] phân tích ba hậu quả nghiêm trọng:

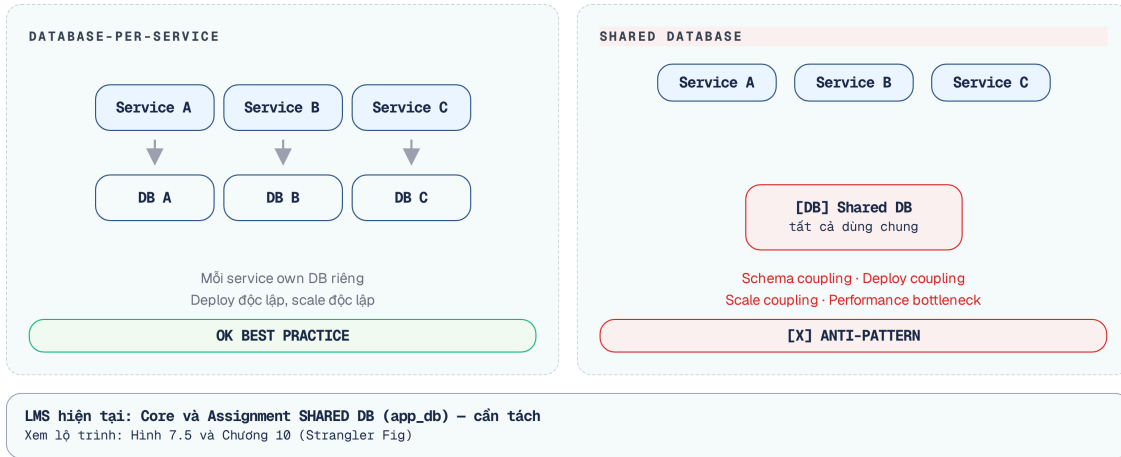


Hình 7.1: Ba hậu quả của shared database trong microservices

- 1. Schema coupling** — Khi hai service đọc cùng bảng `users`, thay đổi schema (đổi tên cột, thêm constraint) ảnh hưởng cả hai. Mỗi lần cập nhật cấu trúc cơ sở dữ liệu (database migration) lại trở thành một bài toán phối hợp liên nhóm phức tạp — đây chính xác là vấn đề mà kiến trúc microservices muốn loại bỏ.
- 2. Deploy coupling** — Database migration phải deploy cùng với tất cả services sử dụng bảng đó. Nếu Service A cần thêm cột vào `users`, Service B phải cập nhật mã nguồn để xử lý cột dữ liệu mới *trước khi* quá trình migration được thực thi — mặc dù Service B hoàn toàn không sử dụng đến trường thông tin đó.
- 3. Scale coupling** — Không thể chọn database technology phù hợp nhất cho từng service. Service yêu cầu full-text search (Elasticsearch) và service yêu cầu ACID transactions (PostgreSQL) bị buộc dùng chung — hoặc phải dùng giải pháp hybrid phức tạp.

7.1.2. Database-per-service pattern

Nguyên tắc **database-per-service** yêu cầu: mỗi service sở hữu dữ liệu riêng, và **service khác không bao giờ truy cập trực tiếp vào database đó** — dữ liệu chỉ được cung cấp ra ngoài qua API hoặc event contract do service sở hữu kiểm soát [2a, Ch.1].



Hình 7.2: Shared Database (đọc/ghi chung) vs Database-per-Service (data qua API)

“Database riêng” không nhất thiết là database server riêng. Có ba cấp độ tách:

Cấp độ	Mô tả	Isolation	Chi phí
Separate schema	Cùng DB server, khác schema/namespace	Thấp (vẫn có thể cross-schema query)	Thấp
Separate database	Cùng DB server, khác database	Trung bình	Thấp
Separate server	Khác DB server hoàn toàn	Cao (cho phép đa công nghệ lưu trữ (polyglot persistence))	Cao

Bảng 7.1: Ba cấp độ tách database

Cấp độ nào phù hợp tùy thuộc vào yêu cầu isolation. Với KBM hiện tại, **separate schema** là bước đầu tiên hợp lý — chi phí thấp nhưng ngăn được schema coupling ngầm.

! Data Should Be Owned, Not Shared

“If a service wants to access data held by another service, it should go and ask that service for the data it needs. And the service exposing the data should decide what to expose and what to hide.”

— Sam Newman, *Monolith to Microservices* [4b]

i KBM shared database

KBM sử dụng **shared database** (`app_db`) giữa Core Service và Assignment Service — cả hai service đọc/ghi cùng PostgreSQL instance, có khả năng truy cập chéo bảng. Đây là hậu quả trực tiếp của việc Assignment được tách ra từ Core (cả hai ban đầu là một monolith `app`). Hậu quả: thay đổi schema cho Assignment có thể ảnh hưởng Core, và ngược lại — phá vỡ independent deployability.

Lộ trình chuyển đổi (Migration path): (1) xác định bảng dữ liệu nào thuộc về Core, bảng nào thuộc Assignment (dựa vào bounded context analysis từ Ch.2), (2) tạo separate schema trong cùng PostgreSQL, (3) thay thế các truy vấn chéo bảng (cross-table queries) bằng các lời gọi API thông qua Feign, (4) dài hạn: tách database server khi cần polyglot hoặc independent scaling.

7.1.3. Industry Case Study: Uber — Domain-Oriented Microservice Architecture (DOMA)

Uber (2020) công bố bài học từ việc quản lý **khoảng 2.200 microservices** — trong đó quản trị phụ thuộc và quyền sở hữu dữ liệu xuyên hàng nghìn service là những thách thức trung tâm. Adam Gluck mô tả cách Uber chuyển từ “microservices spaghetti” sang **DOMA** (Domain-Oriented Microservice Architecture):

Kiến trúc của Uber trải qua ba giai đoạn. Giai đoạn đầu (2012-2016), việc chuyển đổi từ monolith sang microservices diễn ra tự do, dẫn đến gia tăng nhanh chóng số lượng service và phức tạp hóa biểu đồ phụ thuộc.

Trong giai đoạn 2016-2018, hệ thống rơi vào trạng thái “microservices spaghetti” với hàng nghìn dịch vụ giao tiếp và truy cập dữ liệu chéo nhau, phá vỡ tính tự chủ.

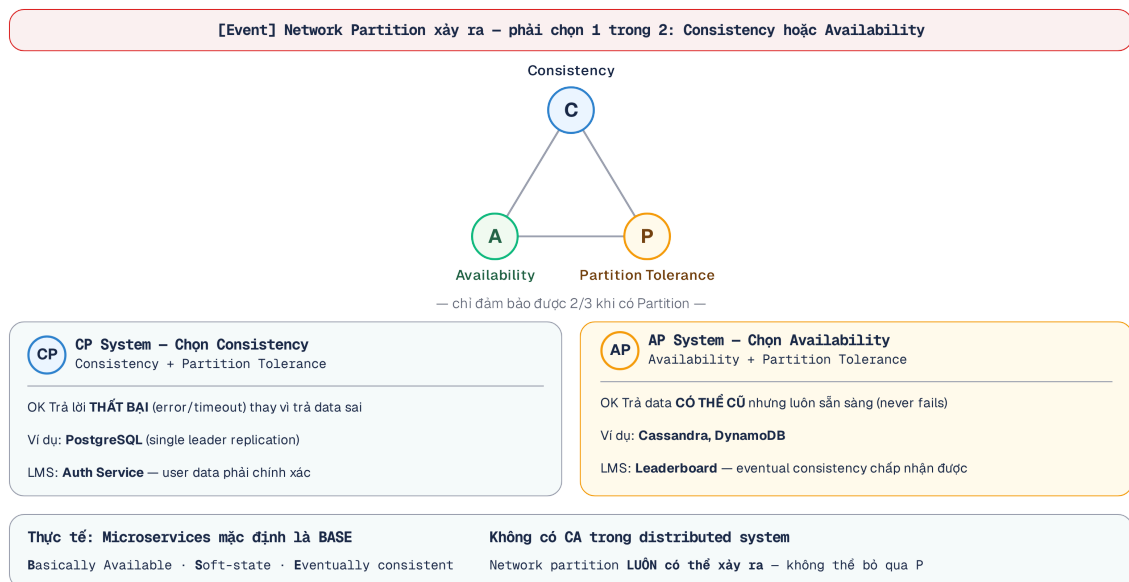
Từ năm 2018, Uber áp dụng DOMA (Domain-Oriented Microservice Architecture) bằng cách gom nhóm các dịch vụ thành những miền (domain) chức năng. Mọi giao tiếp từ bên ngoài bắt buộc đi qua một domain gateway duy nhất, qua đó giảm thiểu sự kết dính dữ liệu.

Bài học rút ra cho data management (giáo trình tổng hợp từ case study trên): (1) Database-per-service là cần thiết nhưng *chưa đủ* — cần thêm *domain-level data encapsulation*. (2) Khi hệ thống lớn, nhóm services thành domains giúp kìm đà tăng của số phụ thuộc chéo. (3) Data duplication giữa domains là chấp nhận được — domain gateway kiểm soát data nào expose.

Nguồn: Adam Gluck, “Introducing Domain-Oriented Microservice Architecture,” *Uber Engineering Blog*, July 2020 (eng.uber.com/microservice-architecture/). Xem thêm: *Uber Technology Day 2019 talks*.

7.1.4. CAP Theorem — Hiểu đúng giới hạn

CAP Theorem được đề cập ngắn gọn ở trên, nhưng hiểu *đúng* CAP rất quan trọng khi thiết kế data management. Kleppmann trong [7, Ch.9] phân tích kỹ: CAP không phải “chọn 2 trong 3” như thường hiểu sai — mà là: **khi network partition xảy ra** (và nó sẽ xảy ra trong distributed systems), hệ thống phải chọn giữa consistency và availability.



Hình 7.3: CAP Theorem — CP (chọn consistency) vs AP (chọn availability) khi partition

CP Systems (Consistency + Partition tolerance): khi partition xảy ra, system ưu tiên consistency — từ chối request thay vì trả data không nhất quán. Ví dụ: PostgreSQL với synchronous replication (quorum commit) đặt consistency lên trên — node không đồng bộ sẽ trả lỗi.

AP Systems (Availability + Partition tolerance): khi partition xảy ra, system ưu tiên availability — vẫn trả response nhưng data có thể stale. Ví dụ: Cassandra, Riak (kiến trúc Leaderless) cho phép đọc/ghi ở bất kỳ node nào — data *eventually* consistent.

Service	Data	Nên CP hay AP?	Lý do
Submission (tạo, chấm)	Transactional	CP	Điểm số yêu cầu độ chính xác tuyệt đối. Việc hệ thống tạm thời từ chối dịch vụ (service unavailable) còn hơn là hiển thị điểm sai, dẫn đến khiếu nại.
Leaderboard	Derived/read	AP	Chấp nhận tính nhất quán cuối. Người dùng thà thấy bảng xếp hạng cập nhật chậm vài giây còn hơn là không thể truy cập dịch vụ.
User profile	Reference	AP	Tên sinh viên rất hiếm khi thay đổi, do đó việc tồn tại dữ liệu cũ (stale data) trong vài phút không gây ra tác động nghiêm trọng
Contest timer	Real-time	CP	Thời gian thi phải chính xác — sai = ảnh hưởng công bằng

Bảng 7.2: CP vs AP cho từng service trong KBM

⚠ CAP Theorem và Network Partition

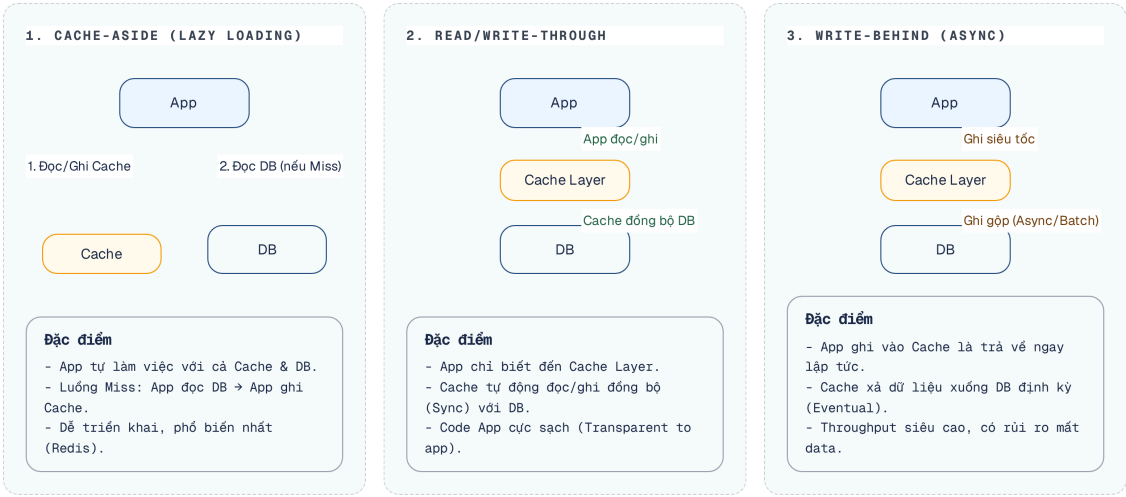
CAP Theorem **không** phải lúc nào cũng hiệu lực. Theorem chỉ áp dụng khi network partition thực sự xảy ra. Trong điều kiện bình thường (không có partition), hệ thống có thể đạt cả Consistency lẫn Availability đồng thời. Hiểu sai điểm này dẫn đến thiết kế quá phòng thủ — chấp nhận eventual consistency ở nơi không cần thiết.

💡 Per-Service CAP Decision

Không phải toàn bộ hệ thống “là CP” hay “là AP”. Mỗi service, thậm chí mỗi *use case*, chọn trade-off riêng. Kleppmann trong [7, Ch.9] gọi đây là “real-world application of CAP”: phân tích *từng data flow* — data nào cần strong consistency (mọi lần đọc thấy bản ghi mới nhất), data nào chấp nhận eventual consistency. Lưu ý phân biệt: chữ C của CAP nói về tính nhất quán giữa các bản sao khi đọc/ghi, khác với chữ C trong ACID (transaction bảo toàn ràng buộc nghiệp vụ) — hai khái niệm trùng tên nhưng không trùng nghĩa.

7.1.5. Caching Strategies — Giảm load và latency

Khi cơ sở dữ liệu đã được phân tách, các truy vấn liên dịch vụ (cross-service queries) sẽ có độ trễ cao hơn do phải thực hiện qua mạng (network call). Sử dụng bộ nhớ đệm (caching) là giải pháp hiệu quả nhất để giảm thiểu độ trễ này, nhưng đòi hỏi chiến lược phù hợp để tránh tình trạng dữ liệu lỗi thời (stale data). Các patterns chính (Newman [4a, Ch.13]):



Hình 7.4: Các caching pattern — Cache-Aside, Read-Through/Write-Through, Write-Behind

Mỗi pattern có ưu nhược điểm riêng và phù hợp với các luồng dữ liệu khác nhau. Việc lựa chọn phụ thuộc vào yêu cầu của từng dịch vụ.

Pattern	Ưu điểm	Nhược điểm	Áp dụng KBM
Cache-Aside	Đơn giản, app kiểm soát	Khi không tìm thấy dữ liệu trong bộ đệm (Cache miss), hệ thống phải thực hiện hai luồng truy vấn (tới cache và database) làm tăng độ trễ	Question list, user profiles
Read-Through	Trong suốt (transparent) đối với ứng dụng	Cache layer phức tạp hơn	—
Write-Behind	Write nhanh (async)	Nguy cơ mất dữ liệu nếu cache gặp sự cố	Bộ đếm view bài giảng (batch 100 views -> DB)

Bảng 7.3: So sánh ba caching patterns

Cache Invalidation — “There are only two hard things in Computer Science: cache invalidation and naming things.” (Phil Karlton). Chiến lược:

Chiến lược vô hiệu hóa bộ đệm (cache invalidation) thường xoay quanh ba phương pháp chính. Đơn giản nhất là sử dụng **TTL (Time-to-Live)**, cho phép dữ liệu tự động hết hạn sau một khoảng thời gian định trước, rất phù hợp với dữ liệu tham chiếu ít thay đổi như danh sách câu hỏi. Đối với dữ liệu biến động liên tục như điểm số, phương pháp **Event-driven invalidation** tỏ ra hiệu quả hơn khi bộ đệm chỉ bị xóa chủ động thông qua các sự kiện phát đi từ hệ thống gốc. Cuối cùng, kỹ thuật **Versioned keys** (đánh dấu phiên bản cho khóa) thường được dùng như một giải pháp dọn dẹp hàng loạt khi có sự thay đổi lớn về cấu trúc dữ liệu.

Trong KBM, Redis hiện đã đảm nhiệm rate limiting ở Gateway (Ch.8); kiến trúc mục tiêu mở rộng vai trò của nó sang caching và leaderboard (mục sau) — hỗ trợ TTL tự động, pub/sub cho invalidation events.

7.2. Chiến lược tách Database từ Monolith

Chia nhỏ cơ sở dữ liệu thường gặp khó khăn tương tự như khi trường đại học tách phòng ban: tách nhân sự (code) thì nhanh chóng, nhưng bóc tách hồ sơ giấy tờ (dữ liệu) lại phức tạp hơn nhiều.

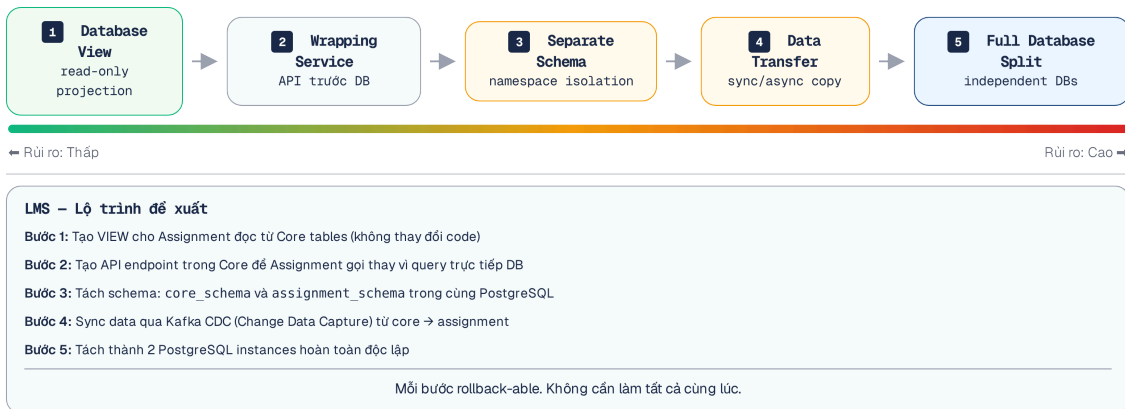
7.2.1. Vấn đề: tách service dễ, tách data khó

Nhiều nhóm phát triển tách microservices ở tầng application (deploy riêng, repo riêng) nhưng vẫn **trở vào cùng database**. Newman trong [4b] gọi đây là “half-baked migration” — nửa vời. Service đã “tách” nhưng data vẫn coupled — lợi ích independent deployability gần như bằng 0.

Tách database là **bước khó nhất** trong quá trình chuyển đổi từ monolith sang microservices. Kleppmann trong [7, Ch.5–6] phân tích: khi data nằm ở nhiều nơi, mọi vấn đề của distributed systems xuất hiện — replication lag, partition tolerance, consistency trade-offs. Đây là chi phí không thể tránh.

7.2.2. Năm chiến lược tách database

Newman đề xuất năm chiến lược, sắp xếp từ ít rủi ro đến nhiều rủi ro [4b, Ch.4]:



Hình 7.5: Năm chiến lược tách database — danh mục tổng quát xếp từ ít đến nhiều rủi ro (nhãn “lộ trình” trong hình mô tả trình tự của danh mục này; KBM chỉ chọn lọc 3 giai đoạn phù hợp — xem bảng dưới)

- 1. Database View** — Tạo view read-only cho service mới, ẩn schema gốc. Ưu điểm: không thay đổi data, service cũ không bị ảnh hưởng. Nhược điểm: chỉ read-only, vẫn coupled vào schema.
- 2. Database Wrapping Service** — Đặt một API đơn giản phía trước database, mọi access đi qua API này. Ưu điểm: bắt đầu tách coupling mà không di chuyển data. Nhược điểm: làm phát sinh thêm một dịch vụ cần được bảo trì (maintain).
- 3. Separate Schema** — Tách bảng vào schema/namespace riêng trong cùng database. Ưu điểm: sự cô lập (isolation) rõ ràng, không yêu cầu thiết lập hạ tầng (infrastructure) mới. Nhược điểm: cùng DB server → vẫn shared resources.
- 4. Data Transfer** — Copy data sang database mới, giữ sync bằng events hoặc CDC (Change Data Capture). Ưu điểm: service mới có data riêng. Nhược điểm: cần cơ chế sync, eventual consistency.
- 5. Full Database Split** — Mỗi service có database server hoàn toàn riêng. Ưu điểm: isolation tối đa, cho phép đa công nghệ lưu trữ (polyglot persistence). Nhược điểm: Đòi hỏi nhiều công sức triển khai và phải xử lý phức tạp các truy cập dữ liệu liên dịch vụ (cross-service data access).

7.2.3. Áp dụng cho KBM

Với bối cảnh KBM (LMS học thuật, nhóm phát triển nhỏ, cùng PostgreSQL), chiến lược phù hợp nhất là tăng dần:

Giai đoạn	Chiến lược	Mô tả	Effort
1	Separate Schema	Tách <code>app_db</code> thành <code>schema core</code> và <code>schema assignment</code>	Thấp
2	API Wrapping	Thay <code>cross-schema queries</code> bằng <code>Feign calls</code>	Trung bình
3	Full Split (nếu cần)	Tách <code>PostgreSQL server</code> khi cần <code>independent scaling</code>	Cao

Bảng 7.4: Migration path tách database cho KBM (3 giai đoạn chọn lọc từ danh mục 5 chiến lược)

💡 Khi nào đủ?

Không phải hệ thống nào cũng cần đến Giai đoạn 3. Với KBM, Giai đoạn 1 (separate schema) đã ngăn được schema coupling ngấm. Giai đoạn 2 (API wrapping) cần khi team mở rộng và cần deploy độc lập thực sự. Giai đoạn 3 chỉ cần khi có yêu cầu scale khác nhau hoặc polyglot persistence.
Chi tiết migration roadmap thực tế, Outbox Pattern (reliable messaging khi tách DB), và thứ tự triển khai → xem Ch.10.

7.3. Data Duplication vs Coupling — Khi nào chấp nhận duplicate?

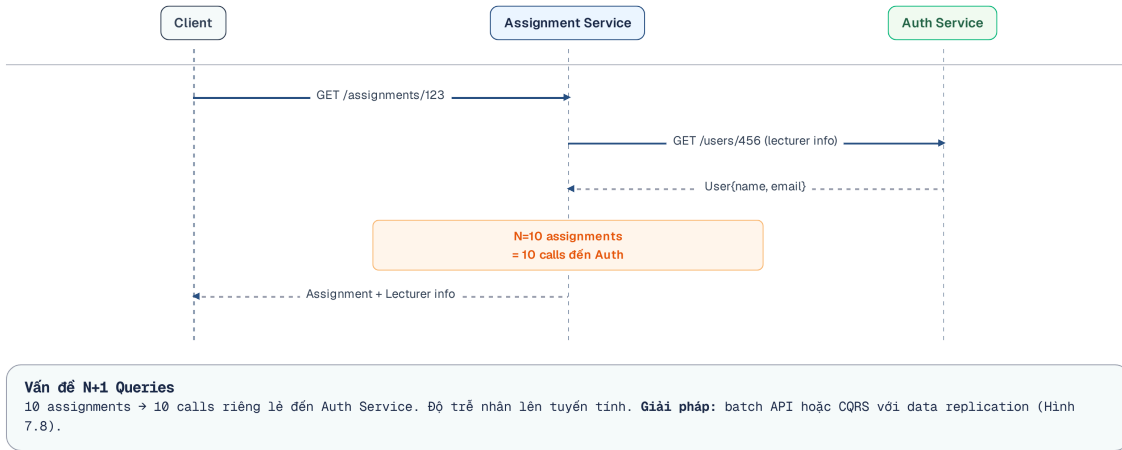
Trong kiến trúc microservices, việc nhân bản dữ liệu thường được áp dụng để đảm bảo tính độc lập. Cách làm này tương tự như việc phòng thư viện lưu bản sao thông tin thẻ sinh viên từ phòng đào tạo để tra cứu nhanh mà không cần đối chiếu chéo liên tục.

7.3.1. Vấn đề: cần data từ service khác

Sau khi tách database, vấn đề tiếp theo lập tức xuất hiện: **service A cần data mà service B sở hữu**. Ví dụ trong KBM: Assignment Service cần hiển thị tên sinh viên — nhưng `users` thuộc về Auth Service. Gọi Auth API mỗi lần cần tên? Lưu copy tên sinh viên tại Assignment?
Mitra trong [3, Ch.5] phân tích ba giải pháp, mỗi cách có trade-off riêng:

7.3.2. Ba chiến lược truy cập data xuyên service

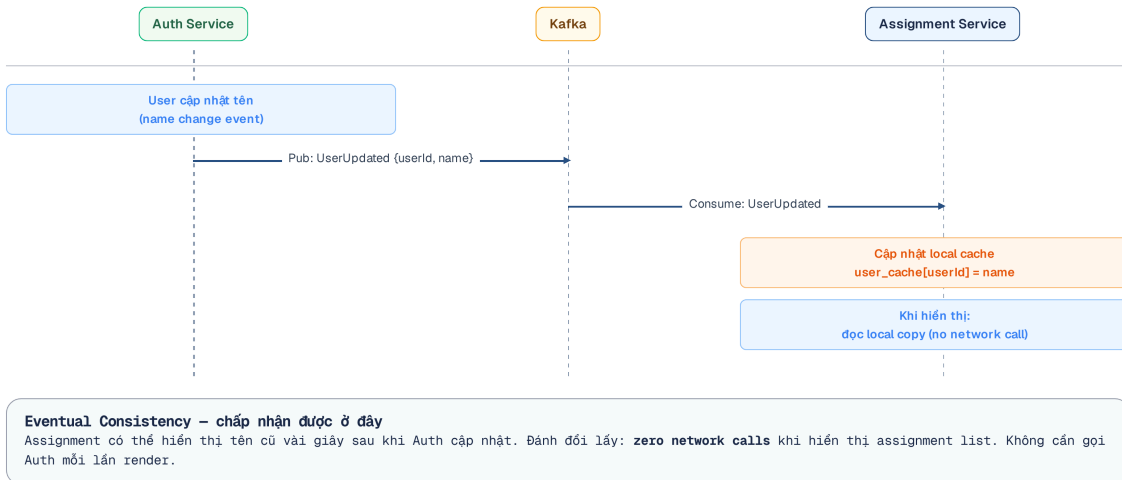
1. API Call (Runtime dependency)



Hình 7.6: Truy cập dữ liệu xuyên service bằng API Call

Sử dụng API Call mang lại ưu điểm rõ rệt là sự đơn giản trong triển khai, không gây trùng lặp dữ liệu và đảm bảo thông tin luôn mới nhất. Tuy nhiên, nó cũng mang theo nhược điểm đáng kể: tạo ra sự phụ thuộc ở thời gian thực (temporal coupling) khiến dịch vụ có thể bị gián đoạn nếu dịch vụ đích gặp sự cố (ví dụ: dịch vụ Auth ngừng hoạt động thì Assignment không hiển thị được tên), đồng thời làm tăng độ trễ mạng và dễ dẫn đến bài toán N+1 queries.

2. Data Duplication (Local copy)

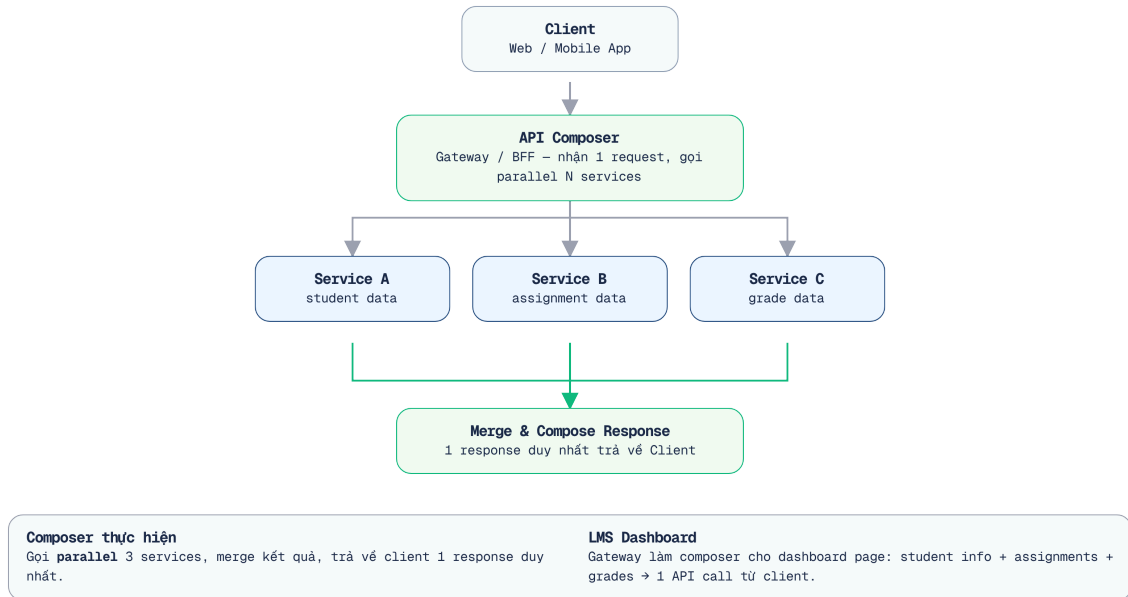


Hình 7.7: Truy cập dữ liệu xuyên service bằng Data Duplication qua Event Pipeline

Ngược lại, Data Duplication khắc phục điểm yếu của API Call bằng cách loại bỏ sự phụ thuộc thời gian thực và cho phép truy vấn dữ liệu cục bộ với tốc độ cao ngay cả khi các dịch vụ khác đang gặp sự cố (downtime). Đổi lại, hệ thống phải chấp nhận tốn thêm dung lượng lưu trữ, đối mặt với tính nhất quán cuối (eventual consistency) và tình trạng dữ liệu lỗi thời (stale data) tạm thời, và tốn công sức xây dựng các luồng đồng bộ sự kiện (event pipeline) phức tạp.

3. API Composition / Data Aggregation

Dùng khi cần join data từ nhiều services. Một service (hoặc API Gateway) gọi nhiều services rồi tổng hợp. Richardson trong [2a, Ch.7] gọi đây là **API Composition pattern**:



Hình 7.8: API Composition pattern — Gateway hoặc BFF gọi nhiều services và tổng hợp

Ví dụ: hiển thị bảng xếp hạng contest cần data từ Core (submissions) và Auth (user names). API Composer gọi cả hai services, sau đó **in-memory join** — thực hiện truy vấn theo lô (batch fetch) danh sách user ID để tránh lỗi N+1 truy vấn, sau đó tổng hợp dữ liệu trên bộ nhớ (in-memory aggregation) để tạo ra `LeaderboardEntry`.

⚠ N+1 Queries và Out-of-Memory

Phương pháp In-memory join hoạt động rất hiệu quả nếu chỉ lấy dữ liệu hiển thị. Tuy nhiên, nếu phát sinh yêu cầu **tìm kiếm/lọc chéo** (ví dụ: “Tìm thứ hạng của sinh viên tên Hùng”), API Composer sẽ phải truy vấn toàn bộ dữ liệu từ Auth (tất cả user tên Hùng) rồi chuyển cho Core để lọc, gây quá tải bộ nhớ và gây tràn bộ nhớ (Out-of-Memory). API Composition chỉ phù hợp khi tiêu chí Search/Sort/Filter nằm trọn trong một Service.

7.3.3. Khi nào chấp nhận duplication?

Newman trong [4b] đưa ra nguyên tắc rõ ràng: “**Duplication is far better than coupling.**” Tuy nhiên, điều này không đồng nghĩa với việc cho phép nhân bản mọi loại dữ liệu một cách tùy tiện. Quy tắc:

Data	Nên duplicate?	Lý do
Reference data (tên user, tên khóa học)	✓ Có	Ít thay đổi, cần hiển thị ở nhiều nơi
Transactional data (điểm số, trạng thái submission)	× Không (Master-Master)	Không duplicate 2 chiều. Copy 1 chiều từ Write Model sang Read Model thì bắt buộc
Configuration (loại database, danh mục)	✓ Có	Gần như static, cache tốt
Audit data (lịch sử thay đổi)	× Không (đa nguồn ghi)	Chỉ một nơi được ghi; bản sao chỉ đọc đưa về Data Warehouse qua ETL/Data Streaming (một chiều) là hợp lệ

Bảng 7.5: Khi nào chấp nhận data duplication

Nguyên tắc quan trọng nhất ở đây là phải luôn biết rõ ai đang là người làm chủ nguồn dữ liệu được sao chép.

Single Source of Truth

Nhân bản dữ liệu (duplication) chỉ chấp nhận được khi tồn tại một **single source of truth** (nguồn sự thật duy nhất) rõ ràng: Service A sở hữu dữ liệu, Service B giữ bản sao. Khi dữ liệu thay đổi, thay đổi chảy theo chiều A → B (qua events), không bao giờ theo chiều ngược lại. Nếu không thể xác định rõ ràng quyền sở hữu đối với một tập dữ liệu, đó là dấu hiệu cho thấy ranh giới ngũ cảnh (bounded context) được thiết kế chưa tốt. Trong trường hợp này, kiến trúc sư cần xem xét lại quá trình phân tích ở Chương 2.

Cross-service queries trong KBM

KBM hiện dùng **Feign calls** để query dữ liệu xuyên service — ví dụ: Assignment Service gọi Core Service để lấy danh sách câu hỏi. Đây là API call pattern (chiến lược 1). Với quy mô học thuật vừa phải, cách này chấp nhận được. Tuy nhiên, trong chế độ thi đấu (contest mode) hoặc trên các bảng điều khiển (dashboard) chứa nhiều thông tin, mỗi lần tải trang có thể kích hoạt (trigger) hàng loạt các lệnh gọi Feign liên tiếp, dẫn đến độ trễ (latency) của hệ thống tăng lên đáng kể.

Migration path: (1) xác định dữ liệu nào Assignment cần *hiển thị* từ Core (chủ yếu reference data: tên câu hỏi, tên cuộc thi), (2) nhân bản dữ liệu tham chiếu (reference data) thông qua các sự kiện Kafka (UserUpdated, QuestionUpdated), (3) giữ Feign calls cho transactional queries (kiểm tra điểm real-time).

7.4. CQRS — Command Query Responsibility Segregation

Hệ thống phân tán có thể tách biệt thao tác ghi và đọc dữ liệu để tối ưu hiệu suất, tương tự như việc trường học tách riêng cổng nộp hồ sơ nhập học (ghi) và bảng tin xem kết quả (đọc).

7.4.1. Từ CQS đến CQRS

Trước khi nói về CQRS, cần hiểu nguồn gốc. Bertrand Meyer đề xuất **CQS** (Command Query Separation) như một nguyên tắc thiết kế ở *cấp độ method*: mỗi method hoặc trả về dữ liệu (query, không thay đổi state) hoặc thay đổi state (command, không trả về dữ liệu) — nhưng không làm cả hai. Rocha trong [5, §4.5.1] phân biệt rõ: **CQS là nguyên tắc ở cấp single-service/single-class**, còn **CQRS mở rộng nguyên tắc**

này lên cấp kiến trúc hệ thống — tách hẳn *model* (thậm chí *database*) cho read và write. CQRS không bắt buộc phải có CQS, nhưng tinh thần tương tự: tách biệt trách nhiệm đọc và ghi.

7.4.2. Vấn đề: cùng model cho read và write không đủ

Trong monolith, cùng một entity **Submission** phục vụ cả hai mục đích:

Write – nộp bài (INSERT), cập nhật kết quả (UPDATE)

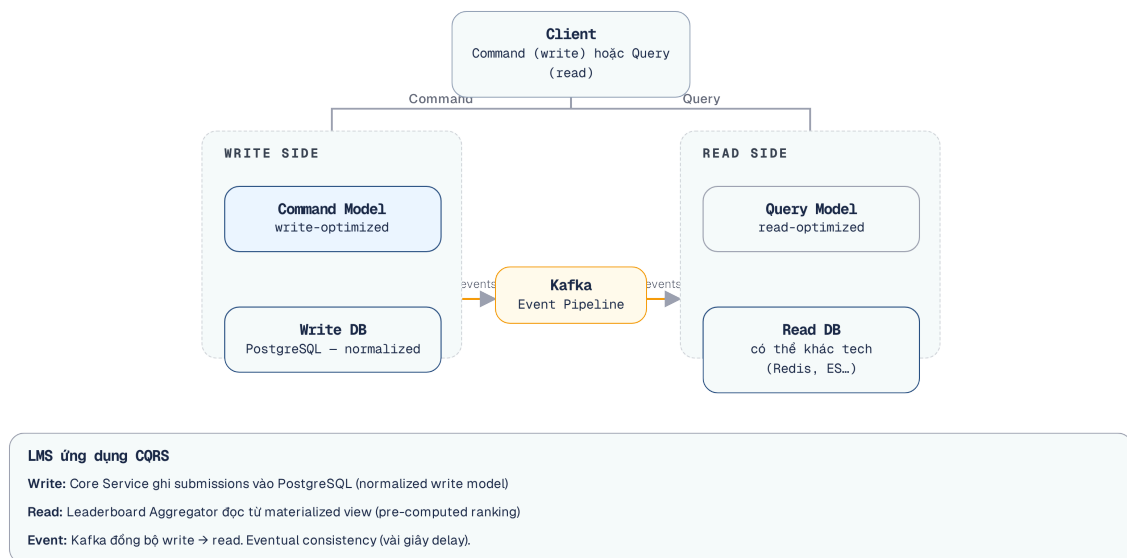
Read – hiển thị lịch sử (SELECT nhiều cột, JOIN nhiều bảng, pagination, filtering)

Yêu cầu của các thao tác đọc và ghi là hoàn toàn khác biệt. Quá trình ghi đòi hỏi tính nhất quán (consistency), khả năng xác thực (validation) và phải tuân thủ chặt chẽ các quy tắc nghiệp vụ (business rules). Trong khi đó, quá trình đọc lại ưu tiên hiệu năng (performance), thường sử dụng cấu trúc dữ liệu phi chuẩn hóa (denormalized data) để hỗ trợ các truy vấn linh hoạt. Khi hệ thống phức tạp, dùng chung một mô hình dữ liệu duy nhất sẽ buộc phải dung hòa cả hai yêu cầu này, khiến không thao tác nào đạt mức tối ưu [2a, Ch.7].

Richardson trong [2a, Ch.7] mô tả vấn đề này đặc biệt nghiêm trọng khi cần **cross-service queries**: hiển thị bảng xếp hạng contest (data từ Core + Judge + Auth) bằng một query duy nhất — *không thể* khi data nằm ở nhiều databases.

7.4.3. CQRS pattern

CQRS (Command Query Responsibility Segregation) tách model thành hai phần riêng biệt [2a, Ch.7]:



Hình 7.9: CQRS pattern — tách command model (write) và query model (read)

Command side (write):

- Normalized data model (đúng 3NF)
- Strong validation, business rules
- Database: PostgreSQL (ACID transactions)

Query side (read):

- Denormalized, pre-joined views
- Tối ưu cho specific query patterns
- Database: có thể khác technology (Elasticsearch cho search, Redis cho leaderboard, PostgreSQL materialized view cho reports)

7.4.4. Khi nào cần CQRS?

CQRS làm tăng độ phức tạp đáng kể — **không sử dụng khi không thực sự cần thiết** [5, §4.5]:

Scenario	Cần CQRS?	Lý do
Tỉ lệ đọc/ghi cân bằng, cùng model	× Không	CRUD đơn giản là đủ
Read phức tạp (cross-service join)	✓ Có	Query model denormalized giải quyết join
Read volume >> Write volume	✓ Có	Cho phép mở rộng quy mô (scale) mô hình đọc độc lập
UI cần real-time views (leaderboard)	✓ Có	Pre-computed views nhanh hơn
Nhóm phát triển quy mô nhỏ, dự án ở giai đoạn thử nghiệm (MVP)	× Không	Chi phí đánh đổi cho sự phức tạp lớn hơn nhiều so với lợi ích thu được.

Bảng 7.6: Khi nào cần CQRS

7.4.5. Ví dụ: Leaderboard trong Contest Mode - Tại sao CQRS bất đồng bộ (qua Kafka) không phù hợp cho Live Leaderboard?

Trong KBM, bảng xếp hạng contest cần join data từ nhiều nguồn:

Bảng xếp hạng cần data từ nhiều services (Auth, Core) — SQL JOIN truyền thống không thể khi database đã tách.

Cách tiếp cận đơn giản nhất là áp dụng CQRS qua Kafka: write side publish `ScoreUpdatedEvent`, read side consume và cập nhật `LeaderboardEntry` denormalized. Tuy nhiên, cách này **không phù hợp về mặt Trải nghiệm người dùng (UX)** cho live contest. Eventual consistency tạo ra replication lag. Hơn nữa, việc áp dụng “Optimistic UI” (tự đoán Rank) là **không khả thi** vì thứ hạng của user phụ thuộc vào điểm của nhiều user khác đang nộp bài cùng lúc.

Giải pháp kiến trúc đề xuất: Sử dụng **Synchronous Materialized View** qua Redis kết hợp với **WebSocket/SSE** thay vì CQRS bất đồng bộ (qua Kafka). Ở đây, Core Service sẽ gọi một **gRPC/REST API đồng bộ** tới Leaderboard Service, sau đó Leaderboard Service tự cập nhật điểm số vào Redis bằng lệnh `ZADD` với kỹ thuật **Composite Score** (ví dụ: `score = (real_score * 1013) - submit_timestamp_ms`, trong đó `submit_timestamp_ms` là mốc thời gian Unix tính bằng *mili-giây*). Composite Score giải quyết bài toán “bằng điểm” (tie-breaker), tránh việc Redis tự động xếp hạng theo bảng chữ cái ABC của `userId`. Redis cung cấp truy vấn thứ hạng cực nhanh qua `ZREVRANK` (ước tính, in-memory), loại bỏ hoàn toàn replication lag cho người nộp bài. Tuy nhiên, để những người dùng khác (observing users) thấy được sự thay đổi ngay lập tức, hệ thống bắt buộc phải tích hợp một cơ chế Server-to-Client Push (như WebSocket hoặc Server-Sent Events) để broadcast sự kiện thay đổi thứ hạng này ra toàn bộ client.

❗ Giới hạn số học của Composite Score

Redis lưu score của Sorted Set bằng số thực dấu phẩy động 64-bit, nên chỉ biểu diễn chính xác các số nguyên trong khoảng $\pm 2^{53} \approx 9,007 \times 10^{15}$. Công thức trên vì vậy đòi hỏi một invariant: `real_score` là số nguyên đã chuẩn hóa theo thang 0–100. Khi đó giá trị composite tối đa khoảng 10^{15} , còn cách tràn an toàn khoảng chín lần. Nếu thang điểm được nới rộng vượt quá 900, phần mili-giây bắt đầu bị làm tròn và tie-breaker mất tác dụng — hai bài nộp cách nhau 1 ms có thể nhận cùng một score. Ngoài ra, hệ số 10^{13} chỉ lớn hơn mốc thời gian Unix mili-giây cho đến năm 2286, và leaderboard cần quy ước rõ giữ *thời điểm sớm nhất* khi một người dùng đạt lại cùng mức điểm.

⚠ Dual-Write (Ghi kép đồng bộ)

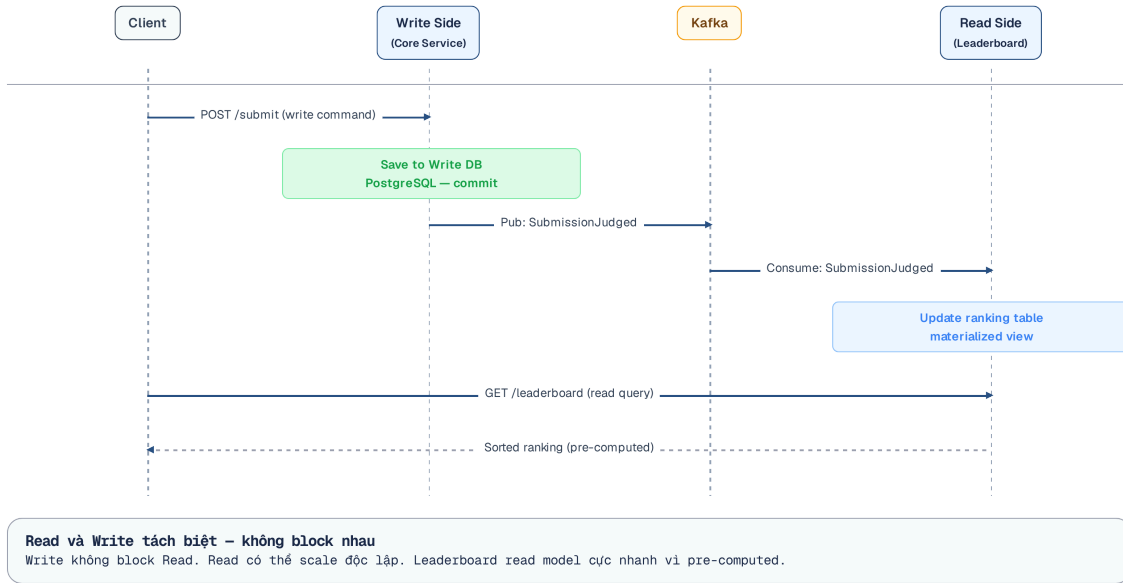
Khi thiết kế Leaderboard cập nhật đồng bộ, tuyệt đối **không cho phép** Core Service truy cập trực tiếp vào cơ sở dữ liệu Redis của Leaderboard Service (phá vỡ Data Encapsulation). Thêm vào đó, việc cập nhật PostgreSQL (DB của Core) và gọi đồng bộ sang Leaderboard Service luôn tiềm ẩn rủi ro **Partial Failure** (lỗi mạng gây lệch dữ liệu).

Để giải quyết dứt điểm, đường bền vững phải là **mặc định** chứ không phải phương án dự phòng: mọi thay đổi điểm được ghi vào **Transactional Outbox (Ch.10) trong cùng local transaction** với PostgreSQL, kèm một `eventId` ổn định. Nhờ đó không tồn tại cửa sổ mất dữ liệu nào, kể cả khi Core crash ngay sau commit.

Lời gọi đồng bộ sang Leaderboard khi đó chỉ là *fast path* để xóa độ trễ ở điều kiện bình thường. Nó mang chính `eventId` nói trên, và Leaderboard xử lý idempotent, nên bản relay từ outbox đến sau không tạo cập nhật trùng — kể cả khi lời gọi đồng bộ bị timeout nhưng thực chất đã thành công. Lưu ý `eventId` chỉ chặn trùng lặp của *cùng một event*, không chặn sai thứ tự giữa *các event khác nhau*: sự kiện phải mang thêm `scoreVersion` tăng dần, và Leaderboard chỉ chấp nhận version mới hơn bản ghi hiện có (so sánh nguyên tử bằng Lua script) — nếu không, bản relay trễ của event cũ có thể ghi đè điểm mới.

Một biến thể cần cảnh giác là “thử sync trước, thất bại mới ghi outbox”. Cách này **không** đóng được cửa sổ dual-write: nếu Core crash sau khi commit database mà chưa kịp ghi outbox, cập nhật mất vĩnh viễn.

Sự khác biệt giữa thiết kế theo hướng đồng bộ bằng Redis so với cách tiếp cận bất đồng bộ thuần túy qua Kafka được thể hiện rõ qua biểu đồ hiệu năng dưới đây.



Hình 7.10: Đường CQRS bất đồng bộ qua Kafka cho Leaderboard — luồng này sinh replication lag; phương án đồng bộ Redis Sorted Set được mô tả trong văn bản

! CQRS ≠ Event Sourcing

CQRS và Event Sourcing thường được nhắc đến cùng nhau, nhưng chúng là **hai pattern độc lập**. CQRS tách read/write model — có thể dùng mà không cần Event Sourcing. Event Sourcing lưu events thay vì state — có thể dùng mà không cần CQRS. Kết hợp cả hai rất mạnh nhưng cũng *rất phức tạp*. Richardson trong [2a, Ch.6] khuyến nghị: bắt đầu với CQRS đơn giản trước, thêm Event Sourcing *khi có nhu cầu cụ thể* (audit trail, temporal queries).

7.5. Event Sourcing — Lưu Events thay vì State

Nếu một bảng điểm chỉ ghi nhận kết quả cuối cùng mà không hề lưu lại quá trình học tập, thi lại hay phức khảo, thì hệ thống đã đánh mất những thông tin lịch sử đặc biệt quan trọng. Event Sourcing ra đời như một cuốn sổ học vụ để lưu trữ lại toàn bộ những biến động này.

7.5.1. Vấn đề: mất lịch sử khi chỉ lưu state

Với database truyền thống, chúng ta lưu **trạng thái hiện tại** (*current state*): `submission.status = JUDGED, score = 85`. Mọi thay đổi trước đó bị ghi đè — không biết submission đã qua những trạng thái nào, ai thay đổi, khi nào.

Trong nhiều domain (tài chính, audit, legal), lịch sử thay đổi có giá trị ngang bằng hoặc hơn trạng thái hiện tại. Ngay cả trong KBM: “sinh viên nộp bài 3 lần, lần đầu sai, lần 2 đúng một phần, lần 3 đúng hoàn toàn” — thông tin này giá trị cho phân tích học tập, nhưng nếu chỉ lưu `status = CORRECT`, lịch sử bị mất.

7.5.2. Event Sourcing pattern

Event Sourcing thay đổi cách lưu trữ: thay vì lưu state, lưu **chuỗi events** (sự kiện) đã xảy ra. State hiện tại được **derive** bằng cách replay tất cả events [2a, Ch.6]:



Hình 7.11: Event Sourcing — lưu chuỗi events thay vì chỉ state hiện tại

Kleppmann trong [7, Ch.11] giải thích: Event Sourcing coi event log là **nguồn sự thật** (source of truth), còn state views (database tables, materialized views) là **derived data** — có thể rebuild bất kỳ lúc nào bằng cách replay events.

💡 Lịch sử biến động và Bảng điểm tổng kết

Các kỹ sư phần mềm đã quen thuộc với mô hình CRUD và câu lệnh **UPDATE** thường gặp khó khăn khi làm quen với Event Sourcing. Hãy nghĩ đơn giản:

Event Store — giống như **Lịch sử biến động điểm/Nhật ký chấm bài** (ghi nhận mọi giao dịch cộng điểm, trừ điểm). Không thể dùng bút xóa sửa lại quá khứ, chỉ có thể ghi thêm sự kiện mới. Nhưng việc lấy nhật ký chấm bài ra để cộng lại xem hiện tại sinh viên đang có bao nhiêu điểm là rất chậm. Hơn nữa, truy vấn kiểu **SELECT * FROM nhap_ky WHERE so_diem_hien_tai > 10** không thực hiện được ngay trên nhật ký — **so_diem_hien_tai** không nằm trong bất kỳ dòng nhật ký nào mà là kết quả *cộng dồn* của cả chuỗi sự kiện (điều này đúng kể cả khi event store được cài trên chính một relational database).

Projection (Materialized View) — giống như **Bảng điểm tổng kết**. Hệ thống tổng hợp toàn bộ dữ liệu sự kiện trong quá khứ, tính toán và xuất ra kết quả “Điểm hiện tại”. Khi UI cần hiển thị, nó chỉ cần đọc bảng điểm này. Trong thực tế, projection gần như luôn cần thiết cho các truy vấn *xuyên aggregate* và cho UI; còn để dựng trạng thái của *một* thực thể, hệ thống chỉ cần replay stream sự kiện của chính thực thể đó.

Ưu điểm	Nhược điểm
Full audit trail — mọi thay đổi đều được ghi	Complexity — rebuild state cần replay, vấn đề về hiệu năng
Temporal queries — “state tại thời điểm T?”	Event schema evolution — events cũ format khác events mới
Hỗ trợ gỡ lỗi (debug) thuận tiện — có thể phát lại (replay) các sự kiện để tái hiện lỗi (reproduce bugs)	Eventual consistency — khi projection cập nhật bất đồng bộ, read model “chậm hơn” event stream và cần xử lý khoảng trễ
Replay — rebuild state, tạo views mới	Storage — event log tăng theo thời gian, cần retention/archiving phù hợp (snapshot chỉ giải bài toán tốc độ replay)
Kết hợp CQRS — events tự nhiên feed vào read models	Learning curve — đòi hỏi sự thay đổi tư duy hoàn toàn so với CRUD

Bảng 7.7: Ưu và nhược điểm của Event Sourcing

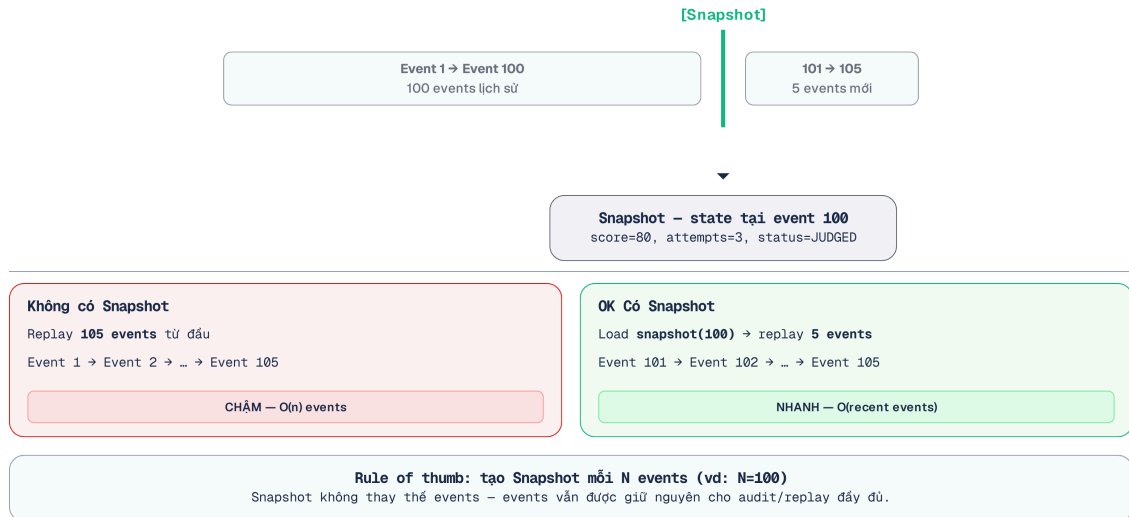
Tuy nhiên, hệ thống Event Sourcing không thể phát lại (replay) toàn bộ sự kiện mỗi khi cần truy vấn trạng thái hiện tại. Điều này tương tự như việc phòng giáo vụ không thể lật lại từng bài kiểm tra từ năm nhất chỉ để tính điểm tích lũy. Giải pháp ở đây là tạo ra các Snapshot (điểm lưu trạng thái).

⚠ Snapshot trong Production

Tiêu chí quyết định snapshot là **độ dài stream của từng aggregate**, không phải tổng số event toàn hệ thống. Một submission chỉ có 5 event thì không bao giờ cần snapshot; nhưng một aggregate sống lâu (ví dụ hồ sơ điểm tích lũy của một sinh viên qua nhiều học kỳ) sẽ khiến mỗi lần dựng lại trạng thái chậm dần theo thời gian. Hãy quyết định dựa trên SLO rehydration đo được — và lưu ý snapshot không giải quyết bài toán dung lượng lưu trữ; đó là việc của retention/archiving.

7.5.3. Snapshot Pattern — Giải quyết Performance của Event Replay

Khi stream của *một aggregate* dài ra (ví dụ hồ sơ điểm của một sinh viên tích lũy hàng nghìn event sau nhiều học kỳ), thời gian dựng lại trạng thái bằng cách replay từ event đầu tiên sẽ **tăng dần theo tuổi đời của aggregate**. **Snapshot pattern** giải quyết:



Hình 7.12: Snapshot pattern — replay từ snapshot thay vì từ đầu

Chiến lược tạo snapshot trong Event Sourcing được thực hiện ở cấp độ **Từng Aggregate (Thực thể)**. Do đó, chiến lược phổ biến và chuẩn mực nhất là dựa trên số lượng (**Count-based**): ví dụ cứ sau 100 events của một thực thể thì tạo một bản snapshot mới.

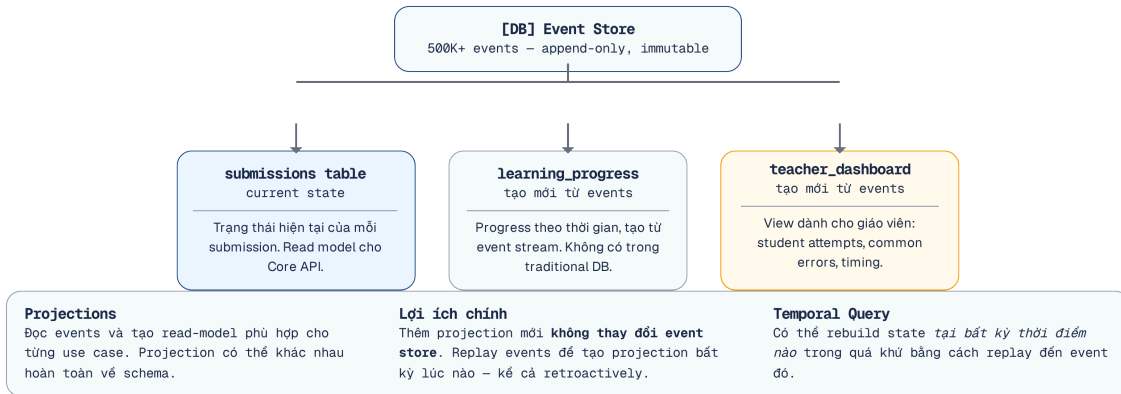
⚠ Performance với Snapshot

Điều cần tránh tuyệt đối là để việc replay không giới hạn nằm trên *request path*: chờ người dùng gọi API GET mới bắt đầu replay hàng trăm nghìn events sẽ gây latency spike nghiêm trọng. Ngoài nguyên tắc đó, count-based (mỗi N event của một aggregate), time-based (theo lịch, ngoài giờ cao điểm — cách LMAX từng dùng) hay on-demand có kiểm soát đều là lựa chọn hợp lệ — chọn theo độ dài stream, tốc độ ghi và SLO phục hồi. Với KBM, count-based chạy nền theo từng aggregate là điểm khởi đầu đơn giản và an toàn nhất.

7.5.4. Projection Rebuilds — Tạo lại Views từ Events

Một trong những lợi ích mạnh nhất của Event Sourcing: **tạo views mới mà không phải sửa event store hay suy diễn lại lịch sử từ current state** (read model mới vẫn cần được tạo schema rồi backfill bằng replay). Ví dụ trong KBM:

1. Tháng 1: chỉ cần bảng `submissions` (score per question)
2. Tháng 6: cần thêm bảng `learning_progress` (trend score theo thời gian per student)
3. Với CRUD: cần database migration + backfill data (khó/không thể nếu data bị ghi đè)
4. Với Event Sourcing: Hệ thống chỉ cần phát lại (replay) toàn bộ sự kiện qua một bộ lọc (projection) mới, từ đó bảng `learning_progress` sẽ tự động được dựng lại cùng với toàn bộ dữ liệu lịch sử (full history)



Hình 7.13: Projection Rebuilds — tạo views mới từ events mà không cần migration

⚠ Idempotency khi build Projection

Khi CQRS tiêu thụ event để build Read Model (Projection), hệ thống message broker (như Kafka) thường chỉ đảm bảo **at-least-once delivery**. Nghĩa là một event có thể bị gửi lặp lại (Duplicate Delivery). Do đó, hàm cập nhật Projection bắt buộc phải có tính idempotent. Cách làm chắc chắn là một bảng inbox/processed-events với ràng buộc `UNIQUE(event_id)`, được ghi *trong cùng transaction* với cập nhật projection — event trùng sẽ vi phạm UNIQUE và bị bỏ qua, tránh cộng dồn sai lệch. Nếu cần chống cả event *sai thứ tự*, so sánh thêm version/sequence của aggregate; đừng dựa vào một mốc `last_processed_event_id` duy nhất, vì `eventId` là định danh duy nhất nhưng không mang thứ tự.

Event Store	Mô tả	Phù hợp khi
EventStoreDB	Database chuyên dụng cho Event Sourcing (by Greg Young)	Cần full ES tính năng: subscriptions, projections, temporal queries
Axon Framework	Java framework cho CQRS + ES, tích hợp Spring	Team Java/Spring, cần opinionated framework
Kafka (with caveats)	Message broker với infinite retention	Đã dùng Kafka, chỉ cần event streaming (không cần query by aggregate)
PostgreSQL + custom	Relational DB với events table	Phù hợp với nhóm phát triển quy mô nhỏ, ưu tiên sự đơn giản và muốn sử dụng hệ quản trị cơ sở dữ liệu sẵn có

Bảng 7.8: Event Store — các công cụ phổ biến

Dù có nhiều lựa chọn, Kafka vẫn thường bị nhầm lẫn với các kho lưu trữ sự kiện thuần túy.

💡 Kafka ≈ Event Store?

Kafka với retention vĩnh viễn (infinite retention) có thể hoạt động như event store. KBM đã dùng Kafka cho submission pipeline (Ch.5). Về lý thuyết, messages trên topic `submissions` và `judge-results` chính là chuỗi events. Tuy nhiên, Kafka được thiết kế là message broker, không phải event store chuyên dụng — querying events theo entity ID khó hơn so với EventStoreDB hoặc Axon. Kleppmann trong [7, Ch.11] phân tích: Kafka phù hợp cho **event streaming** (*process events real-time*), còn Event Sourcing cần **event store** (*query events by aggregate*). Hai use cases liên quan nhưng khác nhau.

7.5.5. Event Schema Evolution — Versioning Events

Events, giống API (Ch.3), cần **schema evolution** khi business thay đổi. Nhưng khác API: events đã lưu *không thể sửa* — event store là append-only. Hai chiến lược:

1. Upcasting — Khi load events cũ, transform sang format mới trước khi apply:

```
// Event v1 (2025): { type: "SubmissionCreated", userId: "123", sql: "SELECT ..." }
// Event v2 (2026): { type: "SubmissionCreated", userId: "123", queryStr: "SELECT ..." }
// Upcaster: xử lý Breaking Change: đổi tên field 'sql' thành 'queryStr'
```

Listing 7.1: Upcasting — transform events cũ sang format mới

2. Weak schema — Events chứa extra fields mà consumer bỏ qua nếu không hiểu (Tolerant Reader — Ch.3). Thêm fields mới → events cũ vẫn valid.

💡 Events là Facts, không phải Commands

Events mô tả điều đã xảy ra (past tense: `SubmissionJudged`), không phải yêu cầu (`JudgeSubmission`). Events immutable — không bao giờ sửa/xóa event đã lưu. Nếu cần “undo”, thêm event mới: `SubmissionResultCorrected`. Richardson trong [2a, Ch.6] nhấn mạnh: đây là *business decision*, không phải *technical decision* — domain experts nên đặt tên events.

7.5.6. Khi nào dùng Event Sourcing?

Scenario	Phù hợp?	Lý do
Audit requirements (tài chính, compliance)	✓ Rất phù hợp	Cần lịch sử đầy đủ, không thể xóa
Temporal analysis (phân tích xu hướng)	✓ Phù hợp	Query “trạng thái tại thời điểm X” tự nhiên
CRUD đơn giản (quản lý users, settings)	× Không	Chi phí vận hành và độ phức tạp phát sinh quá lớn so với lợi ích mang lại
Domain phức tạp (đơn hàng, booking)	✓ Phù hợp	Business events map trực tiếp vào domain events
KBM submission tracking	! Tùy	Có giá trị cho learning analytics, nhưng hiện tại nhóm phát triển nhỏ → không ưu tiên

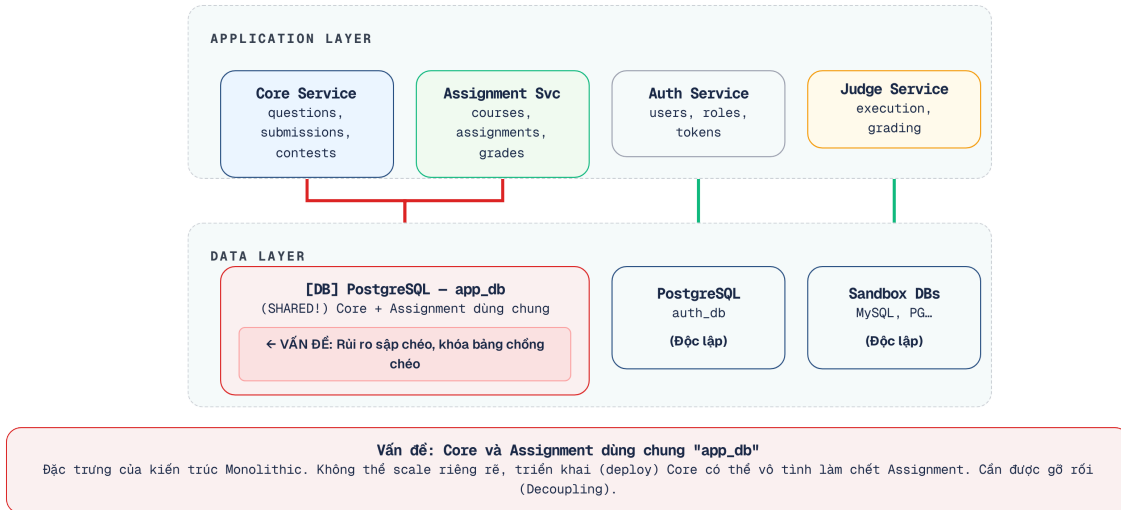
Bảng 7.9: Khi nào dùng Event Sourcing

7.6. Case Study: KBM

Sau khi đã đi qua các nguyên tắc quản lý dữ liệu phân tán, phần này áp dụng chúng vào KBM: nhận diện kiến trúc dữ liệu hiện tại, chỉ ra gap so với best practice, và đề xuất lộ trình chuyển đổi theo giai đoạn.

7.6.1. Hiện trạng

Phân tích source code KBM cho thấy kiến trúc dữ liệu hiện tại:



Hình 7.14: Kiến trúc dữ liệu hiện tại của KBM — shared database là thiếu sót chính

Quan sát chính:

1. **Auth Service** — đã tách database riêng (`auth_db`) → ✓ đúng database-per-service
2. **Core + Assignment** — chia sẻ `app_db` → × vi phạm database-per-service
3. **Judge Service** — không có database riêng, dữ liệu transient → ✓ stateless (OK)
4. **Cross-service data** — dùng Feign calls → ! runtime dependency

Phân tích query patterns: KBM sử dụng hai pattern khi truy vấn dữ liệu vượt ra ngoài một entity:

- **Interface Projections** (Spring Data JPA) — tối ưu truy vấn *nội bộ* (kể cả cross-schema trên shared database hiện tại): trả về lightweight projection thay vì full entity, giảm data transfer. Đây không phải cơ chế vượt service boundary.
- **Feign calls** — pattern cross-service thực sự: gọi API của service khác khi cần data ngoài boundary.

7.6.2. Từ hiện trạng đến kiến trúc mục tiêu

#	Vấn đề	Hiện trạng KBM	Chiến lược migration
1	Shared database	Core + Assignment cùng <code>app_db</code>	Separate schema → Full split
2	Cross-service joins	Cross-table query trực tiếp	Thay bằng Feign hoặc event-based copy
3	Leaderboard query	SQL JOIN trực tiếp	CQRS — pre-computed read model
4	Submission history	CRUD (chỉ lưu current state)	Event Sourcing (cho analytics)
5	User reference data	Feign call mỗi khi cần	Local copy via events
6	Grade Scheme / Attendance / MCQ Daily / CLO	Nhiều read model học vụ cần tổng hợp dữ liệu khóa học, lớp, bài làm, chuẩn đầu ra	Xác định owner rõ ràng, duplicate read-only qua events hoặc materialized views
7	Schema migration không đồng nhất	Java services thiên về JPA auto-DDL; Go services trong DevOps Lab phù hợp hơn với golang-migrate	Chuẩn hóa migration versioned scripts, không dựa vào auto-DDL ở production

Bảng 7.10: Tổng hợp vấn đề data management trong KBM và chiến lược migration

7.6.3. Đề xuất migration path

Lộ trình dưới đây mở rộng 3 giai đoạn tách database ở 7.4 thành kế hoạch data-management đầy đủ: hai phase đầu trùng với giai đoạn 1–2 của bảng; Full Split (giai đoạn 3) chỉ kích hoạt khi cần independent scaling; các phase sau bổ sung phần read model và leaderboard.

Phase 1 — Schema Separation (ưu tiên cao, Chi phí thực hiện thấp): Tách `app_db` thành schema `core_schema` và `assignment_schema`. Để ngăn chặn việc các lập trình viên thực hiện lệnh Cross-Join xuyên schema (gây vô hiệu hóa/lách cơ chế tách dữ liệu), DevOps bắt buộc phải tạo Database Role riêng biệt và cấu hình chặn cứng bằng lệnh `REVOKE ALL ON SCHEMA core_schema FROM assignment_role;`.

Phase 2 — API-based Data Access (Chi phí thực hiện trung bình): Thay cross-schema queries bằng Feign calls — Assignment gọi Core API `GET /api/questions/batch` thay vì query trực tiếp `core_schema.questions`.

Phase 3 — Event-based Data Duplication (Chi phí thực hiện trung bình): Core publish `QuestionUpdated` events khi câu hỏi thay đổi. Assignment consume và lưu local copy (chỉ reference data: title, difficulty). Giảm runtime dependency cho read operations.

Lưu ý Khởi tạo dữ liệu ban đầu (Bootstrapping): Khi áp dụng Data Duplication qua sự kiện, bắt buộc phải thiết kế thêm cơ chế “Bulk Export/Sync” hoặc “Reconciliation” (đối soát định kỳ) để khởi tạo dữ liệu ban đầu khi triển khai (deploy) hoặc khởi tạo môi trường mới. Việc publish event cũng phải đảm bảo tính Atomicity (dùng Transactional Outbox Pattern ở Ch.10) để đề phòng lỗi mạng.

Phase 4 — Real-time Leaderboard với Redis (Chi phí thực hiện cao, giá trị lớn cho contest mode): Tách leaderboard thành read model riêng dùng in-memory data store. Core Service gọi gRPC/REST API tới Leaderboard Service để tự update điểm vào Redis với **Composite Score**. Truy vấn `ZREVRANK` chạy in-memory với độ phức tạp $O(\log N)$ — đủ nhanh cho hiển thị real-time. Ở điều kiện fast path thành công,

người nộp bài đọc lại được ngay điểm vừa ghi (read-after-write trên primary); khi fast path lỗi, bản relay từ outbox vẫn cập nhật bất đồng bộ — tức độ trễ chỉ được loại bỏ trên đường chính, không phải “tuyệt đối”.

Phase 5 — Read models cho learning analytics (khi mở rộng học vụ): Grade Scheme, Attendance, MCQ Daily và CLO nên được xem như read models học vụ. Owner của dữ liệu gốc vẫn nằm trong bounded context tương ứng; bảng tổng hợp chỉ phục vụ query nhanh cho giảng viên và báo cáo, không trở thành nguồn sự thật thứ hai.

⚠ Sai lầm thường gặp

Chia database quá sớm

Tách database trước khi hiểu rõ data access patterns. Hậu quả: phát hiện hai service cần JOIN data liên tục, phải xây dựng luồng dữ liệu sự kiện (event pipeline) phức tạp cho một tác vụ mà trước đây chỉ cần một câu lệnh SQL JOIN đơn giản. **Phòng tránh:** bắt đầu với separate schema (cùng DB server), theo dõi cross-schema queries 2-4 tuần, rồi mới quyết định tách hoàn toàn.

Dùng CQRS cho mọi thứ

Áp dụng CQRS/Event Sourcing cho các chức năng CRUD cơ bản. Hậu quả: Độ phức tạp của hệ thống tăng vọt, đội ngũ phát triển lãng phí thời gian bảo trì luồng sự kiện cho những dữ liệu rất ít khi thay đổi. **Phòng tránh:** Chỉ áp dụng CQRS cho các luồng truy vấn phức tạp (yêu cầu nối dữ liệu liên dịch vụ, lưu lượng đọc lớn, cần giao diện theo thời gian thực). Kiến trúc CRUD cơ bản đã đáp ứng đủ cho phần lớn các trường hợp sử dụng (use cases) thông thường.

Duplicate data nhưng không xác định source of truth

Copy data giữa services mà không rõ “ai sở hữu data này?” Hậu quả: hai services cùng sửa data → conflict, không biết bản nào đúng. **Phòng tránh:** mỗi data entity chỉ có **duy nhất một service sở hữu** — các bản copy khác là read-only replicas, chỉ update qua events từ owner.

Lạm dụng eventual consistency cho Real-time UX

Áp dụng async CQRS (như qua Kafka) cho tính năng cần độ trễ bằng không như Live Leaderboard. Hậu quả: user nộp bài thành công nhưng bảng xếp hạng chưa cập nhật. Optimistic UI không thể áp dụng ở đây vì client không thể tự tính Rank khi nó phụ thuộc vào điểm số của các user khác. **Phòng tránh:** Đối với các trường hợp sử dụng (use case) yêu cầu cập nhật bảng xếp hạng theo thời gian thực (như thi đấu trực tuyến), hãy dùng in-memory data structure cập nhật đồng bộ trên primary (như **Redis Sorted Sets** — cho read-after-write ngay trên đường ghi chính) thay vì async event replication.

💡 Interactive Demo

Xem minh họa động về **Mô hình CQRS và Event Sourcing** tại companion web: cQRS-event-sourcing.html

7.7. Tổng kết

Quản lý dữ liệu là bài toán phức tạp nhất khi chuyển từ monolith sang microservices — phức tạp hơn cả việc tách application code. Database-per-service không chỉ là nguyên tắc kỹ thuật mà là **điều kiện tiên quyết** cho independent deployability — mục tiêu cốt lõi của microservices. CAP Theorem nhắc nhở: trong hệ thống phân tán, consistency và availability luôn phải đánh đổi khi có network partition — mọi quyết định data management đều mang theo trade-off này.

Năm chiến lược tách database (view → wrapping service → separate schema → data transfer → full split) cho phép migration **tăng dần** — không cần “big bang”. Với nhóm phát triển nhỏ như KBM, bắt đầu từ separate schema là bước đi thực tế nhất.

Data duplication không phải anti-pattern — đây là **trade-off có chủ đích** để giảm runtime coupling. Nguyên tắc: mỗi data chỉ có một source of truth, các bản copy là read-only replicas fed bằng events. Newman đã nói rõ: “Duplication is far better than coupling.”

CQRS (mở rộng từ nguyên tắc CQS của Meyer) tách read model và write model — giải quyết bài toán cross-service queries và high-volume reads. Event Sourcing bổ sung bằng cách lưu lịch sử đầy đủ thay vì chỉ state hiện tại. Cả hai pattern này mang lại nhiều lợi ích nhưng rất phức tạp — chỉ áp dụng khi có nhu cầu cụ thể, không phải mặc định.

Một bài toán liên quan chặt chẽ là **distributed transactions** — khi một business operation cần thay đổi data ở nhiều services. Saga pattern (đã thảo luận chi tiết ở Ch.6) là giải pháp: chuỗi local transactions + compensating actions thay vì distributed ACID transaction. Saga và data management hỗ trợ nhau: database-per-service tạo ra nhu cầu Saga; còn Saga có thể dùng chung hạ tầng domain-event/outbox với các projection CQRS hay hệ Event Sourcing — nhưng không phụ thuộc vào hai pattern đó.

Phân tích KBM cho thấy gap nghiêm trọng nhất là shared database giữa Core và Assignment — vi phạm database-per-service và gây deploy coupling. Migration path rõ ràng: separate schema → API wrapping → event-based duplication → CQRS cho leaderboard và learning analytics. Mỗi giai đoạn độc lập, có thể dùng ở bất kỳ giai đoạn nào khi “đủ tốt” cho ngữ cảnh.

Ở Chương 8, chúng ta sẽ chuyển sang **API Gateway** — single entry point cho hệ thống microservices: routing, authentication, rate limiting, và cách KBM sử dụng Spring Cloud Gateway.

Bài tập Chương 7

Xem Phụ lục Bài tập để luyện tập:

- 7-1** – CAP Theorem trong Thực tế — “Pick Two” có đúng không? (Trade-off Analysis [L3])
- 7-2** – Design: Database Migration Strategy cho KBM (System Design [L2])

7.8. Đọc thêm

Sách tham khảo chính:

1. [4b] Sam Newman, *Monolith to Microservices* — Ch.4: Decomposing the Database, Database-per-service patterns
2. [2a] Chris Richardson, *Microservices Patterns*, 1st Ed. — Ch.6: Event Sourcing; Ch.7: Implementing Queries (API Composition, CQRS)
3. [7] Martin Kleppmann, *Designing Data-Intensive Applications* — Ch.5: Replication; Ch.6: Partitioning; Ch.7,9: Transactions & CAP; Ch.11: Stream Processing, Event Sourcing

Sách bổ trợ:

4. [3] Ronnie Mitra & Irakli Nadareishvili, *Microservices: Up and Running* — Ch.5: Data Delegate Pattern, Event Sourcing in practice
5. [5] Hugo Rocha, *Practical Event-Driven MS Architecture* — §4.5: CQS vs CQRS, Command Sourcing; Ch.5: CAP in real world, eventual consistency
6. [6] Eric Evans, *Domain-Driven Design* — Aggregate design → data ownership per bounded context

Chương liên quan:

Ch.6 – Saga Pattern: Distributed transactions, orchestration vs choreography, compensating actions. Saga giải quyết bài toán “thay đổi data ở nhiều services trong một business transaction” — complement trực tiếp cho database-per-service.

Nguồn trực tuyến:

- Martin Fowler, “CQRS” — martinfowler.com/bliki/CQRS.html
- Greg Young, “CQRS Documents” — cqrs.files.wordpress.com
- Confluent, “Event Sourcing with Kafka” — developer.confluent.io

Tài liệu tham khảo

- Kleppmann, M. (2017). *Designing Data-Intensive Applications*. O'Reilly Media.